

Incident Post-Mortem Report

INC-2026-0042: Production Database Outage — Phoenix Platform
Engineering Reliability Team
Severity: P1 Critical | Date: February 18, 2026 | Duration: 2h 47m

1. Incident Summary

Field	Details
Incident ID	INC-2026-0042
Severity	P1 — Critical (complete service outage)
Duration	2 hours 47 minutes (14:23 — 17:10 ICT, February 18, 2026)
Affected Service	Phoenix Platform — core banking application for VietBank
User Impact	Complete outage: 12,000 VietBank end users unable to access online banking, mobile app, and internal teller system
Financial Impact	Estimated VND 450M in lost transaction processing revenue for VietBank. SLA penalty clause triggered: TechViet liable for 2% of monthly contract value (~USD 1,700).
Root Cause	Developer accidentally ran database migration against production instead of staging, causing ACCESS EXCLUSIVE lock on 85M-row transactions table
Detection	Automated (Datadog alert: P95 latency exceeded 30s threshold on /api/v1/transactions)
Incident Commander	Le Van Hung, VP Engineering
Post-Mortem Author	Nguyen Quoc Bao, DevOps Lead

2. Detailed Timeline

Time (ICT)	Event	Actor / System
14:12	Developer (Dev A) opens terminal to run staging migration. Has both staging and production database connection strings in local .env file.	Dev A
14:15	Dev A runs 'alembic upgrade head' — unknowingly connected to production database. The migration contains CREATE INDEX on transactions.created_at column.	Dev A
14:18	PostgreSQL begins building B-tree index on transactions table (85M rows, ~42GB). Acquires ACCESS EXCLUSIVE lock, blocking ALL reads and writes.	PostgreSQL
14:23	Datadog alert fires: P95 latency on /api/v1/transactions exceeds 30s threshold. PagerDuty page sent to on-call engineer (Dev B).	Datadog + PagerDuty
14:25	Dev B acknowledges PagerDuty alert. Checks Datadog dashboard — sees all transaction-related endpoints timing out. Connection pool at 100% utilization.	Dev B (On-call)
14:28	Dev B SSHs into production database monitoring server. Runs pg_stat_activity query — discovers long-running CREATE INDEX with ACCESS EXCLUSIVE lock.	Dev B
14:32	Dev B identifies the migration as unplanned (not in today's deploy manifest). Escalates to P1 severity. Creates #inc-2026-0042 Slack channel.	Dev B

Time (ICT)	Event	Actor / System
14:35	VP Engineering (Le Van Hung) assumes Incident Commander role. DevOps Lead and DBA paged.	IC (Le Van Hung)
14:38	DBA assesses situation: migration is 35% complete. Killing the query would leave a partially-built invalid index requiring manual cleanup (estimated 30+ min).	DBA (Senior)
14:40	IC decision: Allow migration to complete naturally (estimated 40 min remaining). Killing it risks longer total outage and potential data corruption.	IC + DBA
14:45	Account Manager notifies VietBank IT Director via phone. Provides estimated resolution time: 1 hour. VietBank activates their fallback procedures.	Account Manager
15:08	Migration reaches 75%. Datadog shows first signs of some non-transaction endpoints recovering (those not touching the locked table).	Monitoring
15:22	Migration completes. ACCESS EXCLUSIVE lock released. But PgBouncer connection pool remains exhausted — all 200 connections held by stale/retry connections.	PostgreSQL / PgBouncer
15:28	DevOps manually resets PgBouncer connection pool. Connections begin recovering.	DevOps Engineer
15:35	50% of requests succeeding. Application pods health checks passing. Gradual traffic restoration via load balancer.	DevOps + Monitoring
16:00	80% capacity restored. Remaining issues: some user sessions expired during outage requiring re-login.	Monitoring
16:45	100% service restored. All health checks passing. Error rate back to baseline (<0.1%).	Monitoring
17:10	25-minute stability observation period completed successfully. Incident declared RESOLVED.	IC
17:30	Detailed timeline and initial findings shared with VietBank IT team.	IC + Account Manager

3. Root Cause Analysis (5 Whys)

We applied the 5 Whys technique combined with a contributing factors analysis to identify both the immediate cause and the systemic issues that allowed this incident to occur.

Level	Question	Answer
Why 1	Why did the production database lock up for 62 minutes?	A CREATE INDEX operation on an 85M-row table acquired an ACCESS EXCLUSIVE lock, blocking all other operations.
Why 2	Why was CREATE INDEX run instead of CREATE INDEX CONCURRENTLY?	The migration was written by a junior developer and not reviewed for PostgreSQL-specific best practices. Our migration review checklist doesn't include index creation method verification.
Why 3	Why did this migration run against production?	The developer had both staging and production database connection strings in their local .env file and selected the wrong one. There was no confirmation prompt or environment verification.
Why 4	Why does a developer have production database credentials locally?	Our current setup stores all environment configs in a single .env file downloaded from the shared secrets vault. There's no separation between staging and production credentials.
Why 5	Why wasn't this caught by any safety mechanism?	We had no pre-migration safety checks: no table size estimation, no lock impact analysis, no environment confirmation prompt, and no requirement to test against production-scale data.

Contributing Factors

ID	Factor	Category	Severity
F1	Production and staging credentials stored in same .env file with no visual distinction or safety check	Configuration / Credential Management	Critical
F2	No confirmation prompt, environment banner, or guard rail for production database operations	Process / Tooling	Critical
F3	Migration not tested against production-sized dataset (staging has 10K rows vs. 85M in production)	Testing	High
F4	CREATE INDEX used instead of CREATE INDEX CONCURRENTLY — developer was unaware of the distinction	Knowledge / Training	High
F5	PgBouncer connection pool had no circuit breaker or surge protection; exhaustion cascaded the failure	Infrastructure	Medium
F6	Migration code review process does not include DBA review for schema changes on large tables	Process	Medium

4. Action Items

ID	Action	Owner	Priority	Deadline	Status
A1	Separate production credentials into isolated HashiCorp Vault namespace. Developers cannot access production DB credentials without explicit request + DBA approval.	DevOps Lead	P0	Feb 25	DONE

ID	Action	Owner	P r i o r i t y	Deadli ne	Status
A 2	Add production migration safety tool: requires --confirm-production flag, shows environment banner with 10s countdown, logs operator identity.	Platform Team	P 0	Feb 28	DONE
A 3	Engineering standard update: All index operations on tables >1M rows MUST use CREATE INDEX CONCURRENTLY. Added to migration review checklist.	Engineering	P 0	Mar 1	DONE
A 4	Pre-migration analyzer: estimates lock duration based on table size + operation type. Warns if estimated lock >30 seconds. Blocks if >5 minutes.	Platform Team	P 1	Mar 15	IN PROGRESS
A 5	PgBouncer circuit breaker: auto-reset connection pool if utilization stays at 100% for >60 seconds. Implement health-check endpoint for PgBouncer.	DevOps	P 1	Mar 15	IN PROGRESS
A 6	Create production-mirror environment (shadow DB with anonymized production-scale data) for migration testing. Automated nightly refresh.	DevOps + DBA	P 2	Apr 15	PLANNED
A 7	Runbook: database migration rollback procedures for common scenarios (failed index, locked table, bad data migration). Published to Confluence.	DBA Team	P 1	Mar 10	IN PROGRESS
A 8	Training session: safe database migration practices. Mandatory for all engineers. Covers: CONCURRENTLY, lock types, large table strategies, testing.	Engineering + DBA	P 2	Mar 31	PLANNED

5. What Went Well

- Detection was fast: automated monitoring caught the issue within 5 minutes of lock acquisition.
- Incident response was well-coordinated: IC assigned within 12 minutes; clear roles throughout.
- Client communication was timely and transparent: VietBank informed within 22 minutes.
- Post-incident: 3 of 8 action items completed within 10 days. No repeat incidents.

6. What Needs Improvement

- 22-minute gap between lock release and PgBouncer reset — automated pool recovery would have saved this.
- Junior developer had production credentials with no additional safeguards — principle of least privilege violated.
- Migration review process had no DBA checkpoint for schema changes on large tables.
- No production-scale test environment exists — migrations tested only against small staging data.